Lab Assignment 1

Vectors, Matrices, and Linear System Solving

Maths II – Bachelor's Degree in Video Game Design and Development
CITM - UPC

October 2026

Abstract

This assignment introduces a practical workflow for core linear algebra and numerical methods. Students implement custom `Vector` and `Matrix` classes, basic vector-space and matrix-space operations, and a linear-system solver based on Gaussian elimination, back substitution, and residual analysis. The work is validated both mathematically and computationally through an interactive app and automated tests. The goal is to connect formal mathematical definitions with robust C++ implementations, including dimension checks, tolerance handling, and pivoting strategies.

Repository: `https://github.com/jvllahi/math_2_lab`

Exercise 1: Vector and Matrix Methods

a) Mathematical Summary

A vector in $\mathbb{R}^n$ supports:

- Addition: $(u+v)_i = u_i + v_i$
- Subtraction: $(u-v)_i = u_i - v_i$
- Scalar product by $\lambda \in \mathbb{R}$: $(\lambda u)_i = \lambda u_i$
- Dot product: $u \cdot v = \sum_{i=0}^{n-1} u_i v_i$

A matrix in $\mathbb{R}^{m \times n}$ supports:

- Addition/subtraction with same shape
- Scalar product by λ
- Matrix-vector product:

$$(Ax)_i = \sum_{j=0}^{n-1} a_{ij} x_j, \quad i = 0, \ldots, m-1$$

- Matrix-matrix product ($A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$):

$$(AB)_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}$$

Identity matrix I_n is defined by:

$$(I_n)_{ij} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

Suggested pseudocode for matrix-vector product:

```
for i = 0..m-1:
    sum = 0
    for j = 0..n-1:
        sum += A(i,j) * x[j]
    y[i] = sum
```

b) Implementation

Implement methods and operators declared in `LinAlgebra.hpp` and `LinAlgebra.cpp`:

- `Vector`: `operator+=`, `operator-=`, `operator*=`, `dot`
- Free operators for vector unary/binary operations and scalar operations
- `Matrix::identity`
- `Matrix::operator()(row,col)` read/write
- Matrix unary/binary operators, scalar operators, `Matrix*Vector`, `Matrix*Matrix`

NOTE: *Always validate incompatible dimensions and invalid operations.*

c) Report

(a) Which dimension checks are required for each vector/matrix operator? Show one example where an invalid operation must throw an exception.

(b) Compare the computational cost of Ax versus AB using the operation counters (add, subtract, multiply, divide). Provide one concrete example for each case and explain, from the counter results, which operation grows faster when matrix size increases.

(c) Open test `LinearAlgebraTest.AlgebraicExpressionExpApprox` and inspect the implemented expression

$$(I + A + \tfrac{1}{2}A^2)x + \tfrac{1}{4}(Bv).$$

Report: (i) where this test is located, (ii) the numerical value obtained for the result vector with the provided data, and (iii) how you printed that value while running the test.

Exercise 2: Elimination, Back Substitution, and Residual

a) Mathematical Summary

Given $Ax = b$ with $A \in \mathbb{R}^{n \times n}$, eliminate entries below each pivot. At elimination step k:

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \quad i = k+1, \ldots, n-1$$

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik} a_{kj}^{(k)}, \quad i = k+1, \ldots, n-1, \; j = k+1, \ldots, n-1$$

$$b_i^{(k+1)} = b_i^{(k)} - m_{ik} b_k^{(k)}$$

After elimination, solve $Ux = c$ by back substitution:

$$x_i = \frac{c_i - \sum_{j=i+1}^{n-1} u_{ij} x_j}{u_{ii}}, \quad i = n-1, \ldots, 0$$

Residual quality metric:

$$\text{relativeResidual}(A, x, b) = \begin{cases} \dfrac{\|Ax - b\|_2}{\|b\|_2}, & \|b\|_2 \neq 0 \\ \|Ax - b\|_2, & \|b\|_2 = 0 \end{cases}$$

Suggested pseudocode (elimination core):

```
for k = 0..n-2:
    if |A(k,k)| <= tol: return false
    for i = k+1..n-1:
        m = A(i,k) / A(k,k)
        A(i,k) = m
        for j = k+1..n-1:
            A(i,j) -= m * A(k,j)
        b[i] -= m * b[k]
```

b) Implementation

Implement the next functions in `Solver.cpp`:

- `gaussianElimination(Matrix& A, Vector& b, bool pivoting, double tolerance, Operations& operations)`
- `backSubstitution(const Matrix& U, Vector& c, double tolerance, Operations& operations)`
- `relativeResidual(const Matrix& A, const Vector& x, const Vector& b)`

For this exercise, focus on: validating sizes, pivots, multipliers, and row updates.

c) Report

(a) Why can a small pivot produce numerical instability?

(b) Why does back substitution ignore values below the diagonal in U?

(c) Explain why residual operations are not counted in solver operation counters.

(d) Provide one test case where elimination should return `false`.

Exercise 3: Partial Pivoting

a) Mathematical Summary

At each elimination column k, choose:

$$p = \arg \max_{i \in \{k,\ldots,n-1\}} |a_{ik}|$$

If $|a_{pk}| \leq$ tolerance, stop with failure. Otherwise, if $p \neq k$, swap rows k and p in both A and b before elimination updates.

Pivoting reduces sensitivity to round-off and avoids dividing by very small pivots when a better row exists.

Suggested pseudocode:

```
for k = 0..n-2:
    p = argmax_i |A(i,k)| for i in [k, n-1]
    if |A(p,k)| <= tol: return false
    if p != k:
        swap row k and row p in A
        swap b[k] and b[p]
    continue with elimination step
```

b) Implementation

In `Solver.cpp`, extend `gaussianElimination`:

- If `pivoting == true`, choose and swap the pivot row.
- Preserve behavior with `pivoting == false`.
- Keep operation counters only for arithmetic operations, as specified.

c) Report

(a) Compare expected robustness with and without pivoting.

(b) Give one system where no-pivot fails but pivoting succeeds.

(c) Explain the additional computational overhead of pivot search.

(d) How does pivot tolerance interact with pivoting decisions?

How to Test the Implementation

Use the provided presets from the `lab1/` folder.

Assignment implementation

```
cmake --preset default-tests
cmake --build --preset default-tests --parallel
ctest --preset default-tests
```

Also test interactively with the UI. First, compile the executable:

```
cmake --preset default
cmake --build --preset default --parallel
```

Then, run the app:

```
./build/bin/lab1
```

Recommended testing strategy:

- Start with very small systems (2×2, 3×3).
- Add edge cases: incompatible dimensions, near-singular pivots, and zero right-hand side.
- Verify exceptions and boolean failure paths.
- Compare residual quality against expected thresholds in datasets.

Evaluation Criteria

Each exercise has a specific score:

- Exercise 1 (35%): Implementation 25% + Report 10%
- Exercise 2 (35%): Implementation 25% + Report 10%
- Exercise 3 (30%): Implementation 20% + Report 10%

The Lab Assignment is optional. The final Lab grade is computed as the average between this assignment submission and the Lab exam grade.

- If the student does not submit this assignment, the final Lab grade is the exam grade.
- If the student submits this assignment, the average between assignment and exam is used. However, if the average is lower than the Lab exam grade, the Lab exam grade is kept (so the student is not penalized).

Note: The code must work correctly. Otherwise, the grade of the corresponding exercise is zero. If plagiarism is detected, the final grade is zero.

Academic Integrity and AI Use

Using AI tools (LLMs, code assistants) as tutoring support is allowed. Submitting generated code or report text as your own work without disclosure is not allowed.

For submission:

- If AI support was used, include a short file `AI_USAGE.md` in the submitted zip.
- In `AI_USAGE.md`, specify: tool name, what it was used for, and the final prompts that directly influenced the submitted code or report.
- You must be able to explain and justify every submitted function and test.

During review, the student will be asked for a brief explanation of selected code fragments or test decisions. If a submitted part cannot be explained by the student, that part will be considered invalid for grading.

Submission

For submission, you may form groups of up to 3 people. Through ATENEA, use the format:

`Surname1Name1_Surname2Name2_Surname3Name3_Lab1.zip`

with the following content:

- Code files:
  - `LinAlgebra.cpp` with the implementation of classes `Vector` and `Matrix`.
  - `Solver.cpp` with the implementation of the solving methods `Solver`.
- Report (PDF, 2–4 pages).

The submission will be evaluated jointly with the group members. The lab exam will be individual.

Important Dates

Lab1 Assignment Submission Deadline: October 27, 2026.
Term Exam: October 29, 2026.
Lab Practical Exam: November 5, 2026.